

Нейронные сети

Конспект лекции

Автор занятия: Никитин Илья Сергеевич

Дополнительные главы машинного обучения
МОЦП

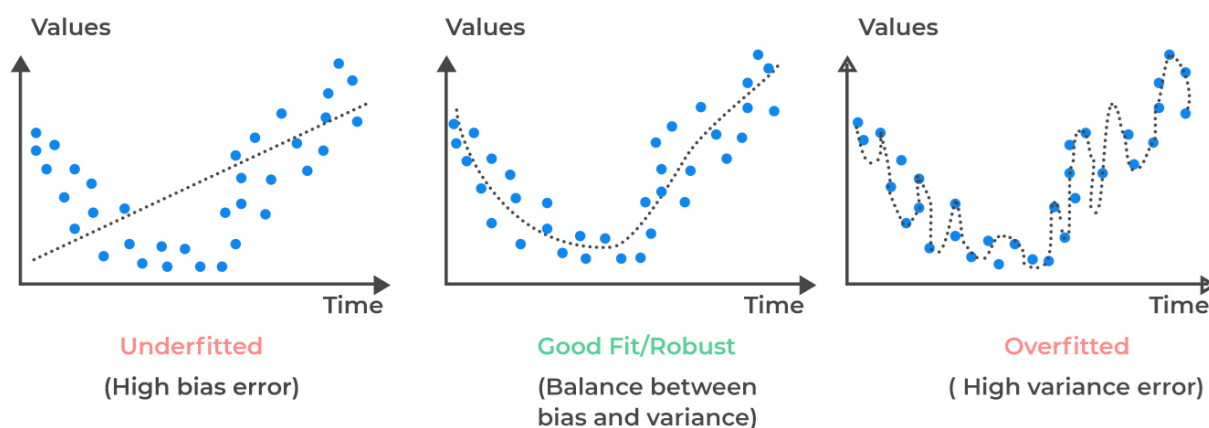
1 Почему линейной модели бывает недостаточно

Прежде чем выбирать способ обучения, стоит выяснить, может ли сама модель описать наблюдаемую зависимость. Рассмотрим линейную регрессию с одним признаком:

$$\hat{y} = wx + b.$$

Она проводит через данные прямую. Если наблюдаемая зависимость заметно искривлена, никакой выбор w и b не позволит прямой повторить её форму. Можно точно решить задачу наименьших квадратов и всё равно получить большую ошибку на тех объектах, по которым модель обучалась.

Это пример *недообучения*: доступное семейство функций слишком ограничено для задачи. Увеличение числа шагов оптимизатора не поможет выйти за пределы этого семейства. Не поможет и дополнительная выборка с той же зависимостью: она уточнит положение лучшей прямой, но не превратит прямую в кривую.



Слева модель слишком проста и пропускает общую форму зависимости. В центре гладкая кривая передаёт эту форму, не повторяя каждое отклонение отдельной точки. Справа модель подстраивается под случайные особенности наблюдений. Рисунок иллюстрирует недообучение и переобучение; качество новой модели нужно проверять на отложенных данных, а не только по виду линии.

У линейной регрессии на небольшом фиксированном наборе простых признаков такая ограниченность встречается часто. Однако линейная модель не обязательно недообучается. При большом числе признаков или после сложных преобразований входа она способна очень точно подогнать train и переобучиться. Например, линейная комбинация степеней $1, x, x^2, \dots, x^d$ линейна по коэффициентам, но уже описывает нелинейную зависимость от x .

1.1 Зачем модели возможность подогнать обучающую выборку

Обучение ищет функцию среди тех, которые допускает модель. Если обозначить это семейство через $\mathcal{F}$, то даже наилучшая доступная ошибка равна

$$\inf_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n \ell(f(x_i), y_i).$$

Если эта величина велика, даже идеальная оптимизация не даст малой ошибки: хорошего решения внутри $\mathcal{F}$ просто нет. Сначала нужно расширить семейство — добавить признаки, нелинейность или скрытые слои. Поэтому важна достаточная ёмкость модели: принципиальная возможность подстроиться под обучающие примеры.

Полезно различать три ситуации. Модель может быть слишком простой и не уметь описать данные; может уметь, но не находить подходящие параметры при выбранном обучении; наконец, может хорошо подогнать `train` и плохо работать на новых объектах. В первом случае меняют представление и архитектуру, во втором разбираются с оптимизацией, в третьем — с обобщением. Одна кривая обучающей ошибки не всегда позволяет однозначно выбрать причину.

Через конечный набор обучающих точек можно провести разные функции. Между наблюдениями и за пределами их диапазона эти функции могут заметно различаться. Поэтому по одному факту точной подгонки нельзя решить, какая из них лучше предсказывает новые ответы.

Начнём со взвешенной суммы, добавим нелинейную активацию, а затем объединим несколько таких преобразований в сеть.

2 Нейрон и нелинейная активация

Пусть объект описан столбцом признаков $x \in \mathbb{R}^d$, а требуется предсказать число. Знакомая линейная регрессия вычисляет

$$\hat{y} = w^\top x + b = \sum_{j=1}^d w_j x_j + b.$$

Коэффициент w_j показывает, насколько изменится ответ при увеличении признака x_j на единицу, если остальные признаки фиксированы. Этот эффект одинаков во всех точках. Например, влияние дополнительного часа подготовки на прогноз результата здесь не зависит от того, сколько часов уже потрачено.

Добавим после взвешенной суммы функцию φ :

$$z = w^\top x + b, \quad h = \varphi(z).$$

Величина z называется *предактивацией*, φ — *функцией активации*, а h — выходом нейрона. Если φ нелинейна, то и зависимость выхода от входа обычно нелинейна. При $\varphi(z) = \max(0, z)$ ответ сначала может оставаться нулевым, а после пересечения порога расти. Смещение b позволяет двигать положение этого порога.

Термин «линейный слой» в машинном обучении часто обозначает преобразование $Wx + b$. Строго математически при $b \neq 0$ оно аффинно. Далее используем привычное название, сохраняя смещение во всех формулах.

2.1 Перцептрон и дифференцируемый нейрон

Исторический пороговый перцептрон выдаёт $\mathbf{1}\{w^\top x + b \geq 0\}$. Это уже нелинейная функция, но её производная равна нулю вне порога, а на пороге не существует. Поэтому обычный градиентный спуск через такую активацию не даёт полезного сигнала. Классический алгоритм обучения перцептрона использует отдельное правило коррекции по ошибкам классификации.

Для градиентного обучения выбирают гладкую активацию либо функцию, дифференцируемую почти всюду, как ReLU. Отдельный нейрон с сигмной обычно называют логистической моделью; многослойный перцептрон, или MLP, состоит из нескольких слоёв таких вычислений. Эти названия родственны, но не означают, что во всех случаях применяется исторический алгоритм перцептрона.

2.2 Нелинейный ответ ещё не означает нелинейную границу

Для бинарного класса удобно получать число от нуля до единицы. Сигмоида переводит произвольный вещественный аргумент в этот диапазон:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = w^\top x + b.$$

Можно интерпретировать p как модельную вероятность класса 1. Но при решении «класс 1, если $p \geq 1/2$ » граница проходит там, где $p = 1/2$. Сигмоида строго возрастает, поэтому

$$\sigma(w^\top x + b) \geq \frac{1}{2} \iff w^\top x + b \geq 0.$$

В двумерном пространстве граница остаётся прямой, в многомерном — гиперплоскостью. Любой фиксированный порог $\tau \in (0, 1)$ также даёт гиперплоскость, только с другим смещением:

$$w^\top x + b = \log \frac{\tau}{1 - \tau}.$$

Вероятность нелинейно зависит от признаков, однако граница разделения остаётся гиперплоскостью.

3 Многослойная полносвязная сеть

Пусть слой получает вектор $h^{(\ell-1)} \in \mathbb{R}^{d_{\ell-1}}$ и выдаёт d_ℓ чисел. Соберём веса нейронов в строки матрицы:

$$z^{(\ell)} = W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}, \quad h^{(\ell)} = \varphi_\ell(z^{(\ell)}), \quad h^{(0)} = x,$$

$$W^{(\ell)} \in \mathbb{R}^{d_\ell \times d_{\ell-1}}, \quad b^{(\ell)}, z^{(\ell)}, h^{(\ell)} \in \mathbb{R}^{d_\ell}.$$

Здесь $W^{(\ell)}$ — матрица весов, а вход $h^{(\ell-1)}$, смещение $b^{(\ell)}$, преактивация $z^{(\ell)}$ и выход $h^{(\ell)}$ — векторы-столбцы. В скрытом слое активация применяется по координатам. Строка j матрицы $W^{(\ell)}$ и элемент $b_j^{(\ell)}$ задают нейрон j . *Полносвязный* слой означает, что каждый выход может использовать каждый вход.

Выходное преобразование выбирается по задаче. Для вещественного ответа, в том числе при приближении x^2 , оставляют линейный выход. Для бинарного класса используют сигмоиду, для одного из нескольких классов — softmax.

3.1 Почему без активаций глубина не помогает

Для двух последовательных аффинных слоёв

$$h = W_1x + b_1, \quad f = W_2h + b_2$$

получаем

$$f = (W_2W_1)x + (W_2b_1 + b_2).$$

Это снова один аффинный слой. Для любого числа таких слоёв рассуждение повторяется. Если между слоями есть узкое место, оно даже ограничивает ранг итоговой матрицы. Факторизация меняет параметризацию и поведение оптимизации, но сама по себе не создаёт нелинейное семейство функций.

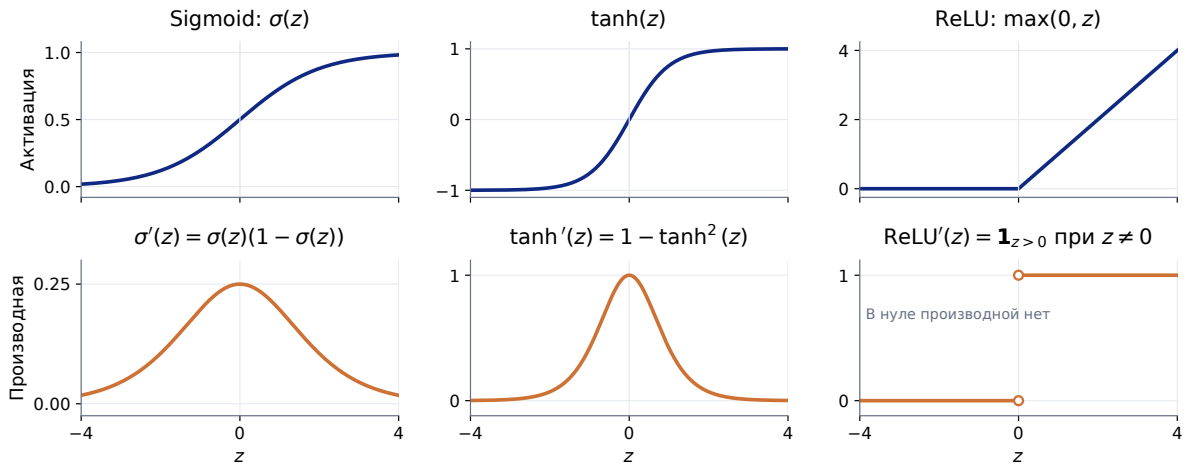
С активацией $W_2\varphi(W_1x + b_1) + b_2$ такое схлопывание в общем случае невозможно. Первый слой формирует признаки, второй комбинирует их. «Обучать представление» здесь означает настраивать и W_1, b_1 , и W_2, b_2 по одной функции потерь.

4 Активации и их производные

Активация влияет и на доступные функции, и на прохождение градиента. Для сигмоиды и гиперболического тангенса

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)), \quad \tanh'(z) = 1 - \tanh^2(z).$$

Когда $|z|$ велико, обе производные близки к нулю: активации входят в насыщение. Сигмоида полезна как выход бинарной вероятностной модели, но в длинной цепочке скрытых слоёв её малые производные могут ослаблять обучающий сигнал.



ReLU определяется как $\text{ReLU}(z) = \max(0, z)$, и

$$\text{ReLU}'(z) = \begin{cases} 0, & z < 0, \\ 1, & z > 0. \end{cases}$$

В нуле ReLU не имеет обычной производной. В вычислениях выбирают соглашение, чаще всего $\text{ReLU}'(0) = 0$. На положительной ветви градиент не сжимается самой активацией. На отрицательной ветви он равен нулю: если нейрон почти всегда получает отрицательные преактивации, его параметры могут перестать обновляться от данных.

Leaky ReLU оставляет небольшой наклон $0 < \alpha < 1$ при $z < 0$: $\max(\alpha z, z)$. Она смягчает проблему полностью неактивного нейрона. Активацию выбирают вместе с инициализацией, масштабированием признаков и скоростью обучения.

Негладкость ReLU обычно не мешает применять градиентные методы: на каждой гладкой области используются обычные производные, а в точках излома — выбранное правило. Но именно около излома численная проверка производных конечными разностями требует осторожности.

5 Выразительность сети: сумма ReLU-признаков

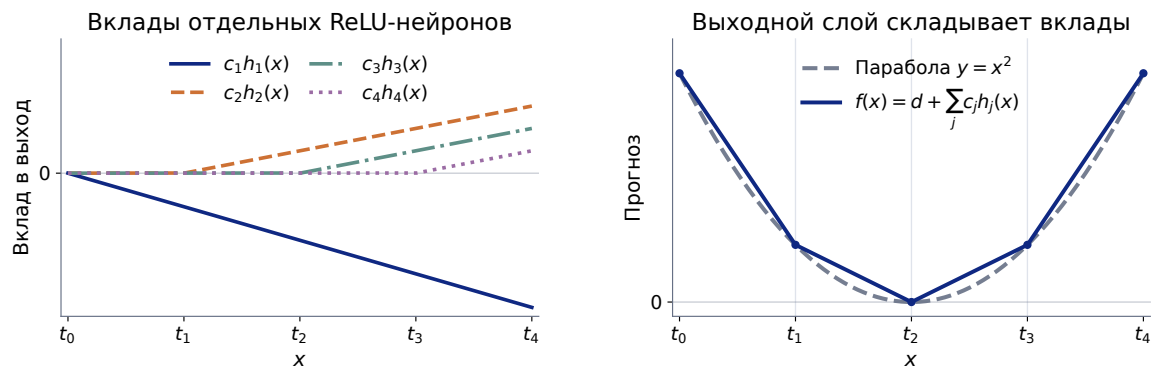
Возьмём один вход x , скрытый слой из m ReLU-нейронов и линейный выход. Первый слой вычисляет m аффинных функций, затем к каждой применяется ReLU:

$$h_j = \text{ReLU}(a_j x + b_j), \quad j = 1, \dots, m.$$

Последний слой складывает эти признаки со своими весами:

$$f_\theta(x) = \sum_{j=1}^m c_j h_j + d = c^\top \text{ReLU}(ax + b) + d.$$

Здесь $a, b, c \in \mathbb{R}^m$, $d \in \mathbb{R}$. Это обычная композиция $\text{Linear} \rightarrow \text{ReLU} \rightarrow \text{Linear}$: все пути от входа к выходу проходят через скрытый слой. Смещения b_j задают положение изломов, входные веса a_j — направление и масштаб каждой ветви, а веса c_j — её вклад в ответ.



Слева показаны отдельные слагаемые $c_j h_j(x)$: это выходы нескольких нейронов *одного* скрытого слоя, умноженные на веса выходного слоя. Справа они сложены и добавлено смещение d . Нейроны работают параллельно; выход одного ReLU не подаётся на вход другого.

При $a_j \neq 0$ выход скрытого нейрона имеет излом там, где $a_j x + b_j = 0$. Поэтому на разных участках оси x сеть может использовать разные наклоны. Несколько таких участков образуют ломаную, а ломаная может приближать параболу. Более широкий скрытый слой позволяет разместить больше изломов и точнее передать кривизну.

Конечная ReLU-сеть остаётся кусочно-линейной: приближение параболы на заданном отрезке не означает точного совпадения на всей числовой прямой.

Рисунок показывает возможности семейства моделей. При обучении положения изломов и наклоны подбираются по данным через веса и смещения. Способность выразить нужную зависимость ещё не гарантирует, что выбранный способ обучения найдёт подходящие параметры.

6 Функции потерь

6.1 От ответа модели к числу, которое нужно уменьшать

Пусть обучающая выборка состоит из пар (x_i, y_i) , $i = 1, \dots, n$. Все обучаемые веса и смещения соберём в θ . Функция потерь одного объекта $\ell(f_\theta(x_i), y_i)$ оценивает расхождение ответа и цели. Эмпирический риск — средняя потеря:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_\theta(x_i), y_i).$$

Функция потерь определяет, какие ошибки штрафуются сильнее. Уменьшение J означает улучшение по этому критерию. К средней потере можно добавить регуляризационный член $\lambda\Omega(\theta)$:

$$J_\lambda(\theta) = J(\theta) + \lambda\Omega(\theta).$$

Здесь $\Omega(\theta)$ — регуляризационный член, а $\lambda \geq 0$ задаёт силу регуляризации. Например, $\Omega = \frac{1}{2} \sum_\ell \|W^{(\ell)}\|_F^2$ штрафует большие веса. Минимизация учитывает одновременно подгонку данных и этот штраф.

6.2 Квадратичная потеря и гауссовская модель

Для вещественного скалярного ответа возьмём

$$\ell_{\text{MSE}}(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2, \quad \frac{\partial \ell}{\partial \hat{y}} = \hat{y} - y.$$

Коэффициент $1/2$ введён для удобства производной. Часто названием MSE обозначают квадрат без него; тогда производная вдвое больше. Без отдельного штрафа минимум не меняется. При фиксированном слагаемом $\lambda\Omega$ изменение масштаба только потерь меняет баланс со штрафом. Нормировку нужно использовать последовательно.

Почему квадрат ошибки естественен? Предположим условную модель

$$y \mid x \sim \mathcal{N}(f_\theta(x), \sigma^2)$$

с фиксированной одинаковой дисперсией. Отрицательный логарифм плотности равен

$$-\log p_\theta(y \mid x) = \frac{(y - f_\theta(x))^2}{2\sigma^2} + \frac{1}{2} \log(2\pi\sigma^2).$$

Последнее слагаемое не зависит от θ , поэтому максимизация правдоподобия эквивалентна минимизации суммы квадратов. Это предположение о шуме, а не утверждение, что реальные ошибки всегда нормальны. Квадрат сильно увеличивает штраф за большие отклонения и потому чувствителен к выбросам.

Для векторного ответа можно использовать $\frac{1}{2} \|\hat{y} - y\|^2$. Усреднение дополнительно по координатам меняет масштаб градиента. Если координаты измеряются в разных единицах, их масштабы или веса нужно согласовать с задачей.

7 Бинарная классификация и кросс-энтропия

Пусть $y \in \{0, 1\}$, а модель выдаёт $p = \sigma(s)$. Число $s \in \mathbb{R}$ называется *логитом*: $s = \log \frac{p}{1-p}$. В модели Бернулли вероятность наблюдаемой метки равна

$$p_\theta(y | x) = p^y(1 - p)^{1-y}.$$

Максимизировать произведение таких вероятностей неудобно: оно быстро становится очень маленьким. Логарифм превращает произведение в сумму, а знак минус превращает максимизацию в минимизацию. Получаем бинарную кросс-энтропию, ВСЕ:

$$\ell_{\text{ВСЕ}}(p, y) = -y \log p - (1 - y) \log(1 - p).$$

При $y = 1$ остаётся $-\log p$: уверенный правильный ответ даёт малую потерю, уверенный неправильный — большую. Логарифмы натуральные; выбор другой базы лишь умножил бы потери на константу.

Подставим $p = \sigma(s)$ и перегруппируем слагаемые:

$$\ell_{\text{ВСЕ}}(s, y) = \log(1 + e^s) - ys = \text{softplus}(s) - ys.$$

Прямое вычисление e^s при большом s может переполниться. Устойчивая эквивалентная запись имеет вид

$$\ell_{\text{ВСЕ}}(s, y) = \max(s, 0) - ys + \log(1 + e^{-|s|}).$$

Экспонента теперь применяется только к неположительному числу. На практике предпочтительна функция потерь, принимающая логиты: она объединяет сигмоиду и логарифмы в устойчивое вычисление. Предварительно применять сигмоиду к её аргументу не нужно.

7.1 Почему производная имеет вид $p - y$

По правилу цепочки

$$\begin{aligned} \frac{\partial \ell}{\partial p} &= -\frac{y}{p} + \frac{1-y}{1-p}, & \frac{\partial p}{\partial s} &= p(1-p), \\ \frac{\partial \ell}{\partial s} &= p - y. \end{aligned}$$

При уверенном неправильном ответе $p \approx 0$, $y = 1$ производная по логиту близка к -1 , а не к нулю. Это результат сочетания ВСЕ с сигмоидой. Если вместо ВСЕ взять квадрат ошибки вероятности, в производной останется дополнительный множитель $p(1-p)$, ослабляющий сигнал в насыщении.

8 Несколько классов и softmax

Если у объекта ровно один из K классов, сеть выдаёт логиты $s \in \mathbb{R}^K$. Преобразование softmax задаёт вероятности

$$p_k = \frac{e^{s_k}}{\sum_{j=1}^K e^{s_j}}, \quad p_k > 0, \quad \sum_{k=1}^K p_k = 1.$$

Пусть $y \in \mathbb{R}^K$ — one-hot-вектор: единица стоит у правильного класса c . Отрицательное логарифмическое правдоподобие категориального распределения даёт

$$\ell_{\text{CE}}(s, y) = - \sum_{k=1}^K y_k \log p_k = -s_c + \log \sum_{j=1}^K e^{s_j}.$$

При $m = \max_j s_j$ вычисляем

$$\log \sum_j e^{s_j} = m + \log \sum_j e^{s_j - m}.$$

Это приём log-sum-exp: экспоненты не переполняются. Softmax не меняется при прибавлении общей константы ко всем логитам. Значимы их различия, а не абсолютный уровень.

Производная потери по каждому логиту равна

$$\frac{\partial \ell}{\partial s_k} = p_k - y_k.$$

Вывод использует $\partial \log \sum_j e^{s_j} / \partial s_k = p_k$. Нормировка по n объектам даёт дополнительный множитель $1/n$ в градиенте общей потери. В задаче нескольких независимых меток на объект вместо softmax обычно нужны отдельные сигмоиды и BCE по меткам: сумма вероятностей тогда не обязана равняться единице.

8.1 Потеря и метрика отвечают на разные вопросы

Ассигасу сравнивает дискретные предсказанные классы с ответами. Если уверенность в правильном классе растёт, но предсказанный класс остаётся прежним, ассигасу не меняется, а SE уменьшается. Для оптимизации полезен чувствительный к параметрам критерий; для оценки задачи могут быть важны ассигасу, полнота редкого класса, MAE или качество калибровки вероятностей.

9 Градиентный спуск

9.1 Почему двигаются против градиента

Теперь архитектура и $J(\theta)$ заданы. Нужно найти параметры с малой потерей. Градиент $g = \nabla_{\theta} J$ содержит производные по всем параметрам. Для малого изменения $\Delta\theta$ первое приближение имеет вид

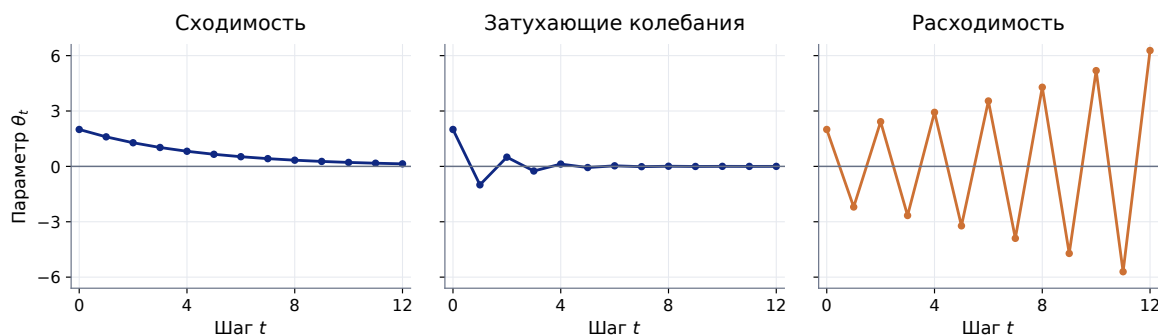
$$J(\theta + \Delta\theta) \approx J(\theta) + g^{\top} \Delta\theta.$$

Если выбрать $\Delta\theta = -\eta g$ при $\eta > 0$, линейный член равен $-\eta \|g\|^2 \leq 0$. Это даёт правило градиентного спуска:

$$\theta_{t+1} = \theta_t - \eta_t \nabla J(\theta_t).$$

Скорость обучения, learning rate η_t , задаёт масштаб шага. Градиент показывает локальное направление, но не говорит, насколько далеко в нём можно идти. При слишком большом шаге линейное приближение перестает быть точным, и потеря может вырасти.

Для $J(w) = \frac{1}{2}aw^2$, $a > 0$, итерация равна $w_{t+1} = (1 - \eta a)w_t$. Сходимость к нулю происходит при $|1 - \eta a| < 1$, то есть $0 < \eta < 2/a$. При малом шаге движение медленно, при промежуточном возможно колебание через минимум, при слишком большом — расхождение.



У многомерной задачи разные направления могут иметь разную кривизну. Шаг, подходящий для пологого направления, оказывается слишком большим для крутого. Масштабирование признаков и подходящая параметризация часто улучшают эту геометрию.

10 Стохастическое обучение и мини-батчи

Полный градиент равен среднему градиентов отдельных объектов:

$$\nabla J(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla_{\theta} \ell_i(\theta).$$

Вычислять его перед каждым шагом дорого для большой выборки. Вместо этого выбирают мини-батч $\mathcal{B}_t$ из B объектов и используют

$$g_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla_{\theta} \ell_i(\theta_t), \quad \theta_{t+1} = \theta_t - \eta_t g_t.$$

При $B = 1$ получают стохастический градиентный спуск, SGD; при $B = n$ — полный градиентный спуск. Термином SGD часто называют и мини-батчевый вариант. Один *шаг* обрабатывает батч, одна *эпоха* — полный проход по обучающим объектам. При разбиении выборки на непересекающиеся батчи без отбрасывания остатка эпоха содержит $\lceil n/B \rceil$ шагов.

Для равномерного независимого выбора объектов при фиксированном θ имеем $\mathbb{E}[g_t] = \nabla J(\theta)$. Здесь J — среднее потерь независимых прямых проходов; при штрафе $\lambda \Omega$ к оценке добавляют $\lambda \nabla \Omega$. При независимом выборе с возвращением ковариация среднего убывает как $1/B$, без возвращения есть поправка на конечную выборку. Большой батч даёт менее шумное направление, но требует больше работы на шаг и памяти. В проходе с перемешиванием оценки между шагами не независимы. Обучающий BatchNorm связывает объекты, поэтому для него такая простая интерпретация среднего отдельных градиентов требует изменения.

Если заменить среднюю потерю батча суммой, градиент того же батча увеличится в B раз. Для простого SGD это равносильно увеличению шага в B раз; для momentum и адаптивных методов нужно учитывать также внутреннее состояние и детали обновления. Поэтому нормировка потери является частью алгоритма обучения.

Из-за шума потеря отдельного батча не обязана монотонно уменьшаться. Сравнивать динамику разумнее по средним и по отдельной валидации. Небольшой случайный подъём не доказывает ошибку; устойчивый рост или нечисловые значения требуют проверки.

10.1 Невыпуклость и критерий остановки

Линейная регрессия с квадратом ошибки имеет выпуклую задачу по своим параметрам. В многослойной сети композиции и произведения весов обычно делают задачу невыпуклой. Возможны седловые точки, плоские области, много различных наборов весов с похожей потерей. Обычный градиентный спуск не обещает найти глобальный минимум.

Кроме того, минимум обучающей потери не равен наилучшему качеству на новых данных. Момент остановки выбирают по валидации, сохраняют лучшую модель и проверяют качество отдельно. Нулевая обучающая ошибка сама по себе не подтверждает обобщение.

11 Momentum и Adam

Градиенты соседних батчей могут заметно различаться. Momentum накапливает их экспоненциальное скользящее среднее:

$$m_t = \beta m_{t-1} + (1 - \beta)g_t, \quad \theta_{t+1} = \theta_t - \eta_t m_t, \quad m_0 = 0.$$

Коэффициент $\beta \in [0, 1)$ определяет память. Если раскрыть рекурсию, вклад градиента g_k в m_t будет равен $(1 - \beta)\beta^{t-k}g_k$. Старые направления постепенно забываются. Согласованные градиенты поддерживают движение, а часть колебаний между батчами усредняется. Есть соглашение без множителя $(1 - \beta)$ перед g_t ; при нём масштаб накопленного вектора и смысл learning rate отличаются.

11.1 Adam: направление и масштаб для каждой координаты

Adam [7] хранит два экспоненциальных средних: самих градиентов и их квадратов. На шаге $t \geq 1$ градиент текущего батча g_t вычисляется при параметрах θ_{t-1} . Начальное состояние $m_0 = v_0 = 0$, а накопители обновляются так:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2)(g_t \odot g_t).$$

Первый накопитель задаёт усреднённое направление. Второй оценивает величину квадратов градиентов по каждой координате. Это оценка второго начального момента, а не дисперсии: квадрат среднего из v_t не вычитается. Обычно $\beta_1, \beta_2 \in [0, 1)$ выбирают разными, поскольку направление и масштаб усредняют на разных горизонтах.

Из-за нулевого начального состояния сумма весов среднего на шаге t равна $1 - \beta^t$, а не единице. Поэтому Adam выполняет поправку на начальное смещение:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

Например, при неизменном среднем градиента имеем $\mathbb{E}[m_t] = (1 - \beta_1^t)\mathbb{E}[g_t]$; деление устраняет именно влияние нулевой инициализации. Оно не делает меняющийся во время обучения градиент стационарным.

Обновление параметров имеет вид

$$\theta_t = \theta_{t-1} - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

Квадрат, корень и деление здесь покоординатные. Координата с большим накопленным квадратом градиента получает меньший масштаб шага при том же числителе. Положительное ε предотвращает деление на ноль и ограничивает усиление там, где $\hat{v}_t$ очень мало; оно также влияет на шаг в этом режиме. Градиенты для Adam по-прежнему вычисляет backprop. Общий learning rate и качество на валидации всё равно нужно контролировать.

12 Организация обучения и проверки

До обучения отделяют train, validation и test. По train оценивают параметры преобразования данных и веса сети. По validation выбирают архитектуру, регуляризацию, learning rate и момент останова. Test используют для итоговой оценки после выбора модели. Если многократно подбирать решение по test, он фактически превращается в validation и перестаёт давать независимую проверку.

Обычный цикл одного шага состоит из следующих действий:

1. Получить батч и выполнить прямой проход сети.
2. Вычислить скалярную потерю с согласованной нормировкой.
3. Обнулить старые накопленные градиенты и выполнить backward.
4. Обновить все обучаемые параметры выбранным оптимизатором.

Накопление градиентов нескольких батчей тоже возможно, но тогда это намеренная часть алгоритма с отдельным учётом нормировки. На валидации параметры не обновляют, а слои с разным поведением при обучении и применении переключают в соответствующий режим.

Для временного ряда разделение должно соответствовать будущему применению. Случайно разделённые перекрывающиеся окна могут передать информацию между обучением и проверкой. При прогнозировании будущего используют разделение по времени; при нескольких независимых объектах дополнительно учитывают группы. Нормировки и другие обучаемые преобразования оценивают только на train.

13 Обратное распространение ошибки

13.1 Начинаем с правила цепочки

Градиентный спуск требует производную потерь по каждому весу. Даже небольшая сеть — композиция многих операций. Выписывать огромную формулу и отдельно дифференцировать её по каждому параметру неудобно. Вакрогг применяет правило цепочки к вычислительному графу и использует промежуточные результаты повторно.

Пусть один нейрон вычисляет

$$z = wx + b, \quad h = \varphi(z), \quad \ell = \frac{1}{2}(h - y)^2.$$

При прямом проходе получаем z, h, ℓ . При обратном сначала узнаём, как потеря зависит от выхода:

$$\frac{\partial \ell}{\partial h} = h - y.$$

Затем переносим эту чувствительность к предактивации:

$$\delta = \frac{\partial \ell}{\partial z} = \frac{\partial \ell}{\partial h} \frac{\partial h}{\partial z} = (h - y)\varphi'(z).$$

Теперь локальные производные аффинной операции дают

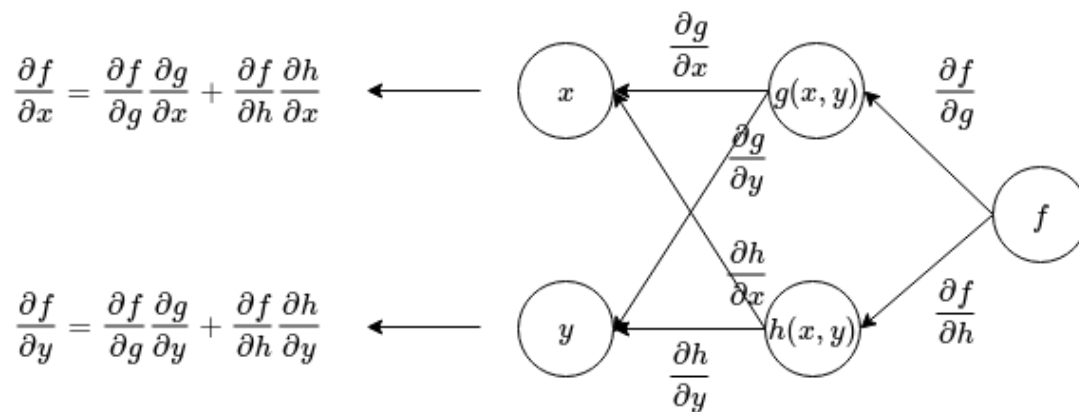
$$\frac{\partial \ell}{\partial w} = \delta x, \quad \frac{\partial \ell}{\partial b} = \delta, \quad \frac{\partial \ell}{\partial x} = \delta w.$$

Одна уже вычисленная величина δ нужна сразу для трёх результатов. Название «распространение ошибки» относится к передаче таких производных от выхода к входу. Это не пересылка самой численной потери назад и не отдельное правило изменения весов.

13.2 Ветвления требуют сложения

Пусть $g = g(x, y)$ и $h = h(x, y)$, а итоговая функция равна $f = F(g, h)$. Переменная x влияет на f двумя путями: через g и через h . Полная производная должна учесть оба:

$$\frac{\partial f}{\partial x} = \frac{\partial F}{\partial g} \frac{\partial g}{\partial x} + \frac{\partial F}{\partial h} \frac{\partial h}{\partial x}.$$



Стрелки на схеме направлены от результата к аргументам. На каждом ребре градиент умножается на локальную производную, а у общей переменной вклады суммируются. Аналогичная формула получается для y . Пересечение линий не является отдельной операцией. Такое сложение понадобится при остаточном соединении, повторном использовании веса и суммировании потерь объектов.

14 Обратный проход для слоёв и мини-батчей

Пусть $z = Wh + b$, где $W \in \mathbb{R}^{m \times d}$, $h \in \mathbb{R}^d$, $z, b \in \mathbb{R}^m$. Из следующих операций уже пришёл столбец $\delta = \partial\ell/\partial z \in \mathbb{R}^m$. Координатно

$$z_j = \sum_{k=1}^d W_{jk} h_k + b_j, \quad \frac{\partial\ell}{\partial W_{jk}} = \delta_j h_k, \quad \frac{\partial\ell}{\partial h_k} = \sum_{j=1}^m W_{jk} \delta_j.$$

Отсюда следуют три правила:

$$\nabla_W \ell = \delta h^\top, \quad \nabla_b \ell = \delta, \quad \nabla_h \ell = W^\top \delta.$$

Внешнее произведение $\delta h^\top$ имеет размер $m \times d$, как матрица весов. Транспонированная матрица $W^\top \in \mathbb{R}^{d \times m}$ возвращает чувствительность к d входам. Проверка размеров помогает обнаружить неправильное транспонирование ещё до вычислений.

Для покоординатной активации $h = \varphi(z)$ при входящем градиенте $g = \partial\ell/\partial h$ имеем

$$\frac{\partial\ell}{\partial z} = g \odot \varphi'(z).$$

Символ $\odot$ обозначает покоординатное произведение. Якобиан такой активации диагонален, поэтому строить полную матрицу не требуется. Для softmax якобиан уже не диагонален: изменение одного логита меняет все вероятности. Именно поэтому удобно использовать уже выведенную общую производную softmax вместе с СЕ.

14.1 Правило для всей последовательной сети

Обозначим $\delta^{(\ell)} = \partial\ell/\partial z^{(\ell)}$. Если выходной градиент известен, скрытые слои обходят в порядке $L-1, \dots, 1$:

$$\delta^{(\ell)} = (W^{(\ell+1)\top} \delta^{(\ell+1)}) \odot \varphi'_\ell(z^{(\ell)}),$$

$$\nabla_{W^{(\ell)}} \ell = \delta^{(\ell)} h^{(\ell-1)\top}, \quad \nabla_{b^{(\ell)}} \ell = \delta^{(\ell)}.$$

Прямой проход сохраняет нужные h и z ; обратный использует их повторно. При фиксированной архитектуре его вычислительная стоимость обычно того же порядка, что и прямого прохода, хотя константы и расход памяти различаются.

14.2 Обратное распространение для мини-батча

Пусть столбцы $H^{(0)} = X \in \mathbb{R}^{d_0 \times B}$ — объекты батча. Для слоя

$$Z^{(\ell)} = W^{(\ell)} H^{(\ell-1)} + b^{(\ell)} \mathbf{1}_B, \quad H^{(\ell)} = \varphi_\ell(Z^{(\ell)}).$$

Вектор $\mathbf{1}_B$ из B единиц копирует одно смещение во все столбцы. Матрицы $Z^{(\ell)}, H^{(\ell)}$ имеют размер $d_\ell \times B$. Веса и смещения общие для всех объектов.

Возьмём *среднюю* потерю $J_B = \frac{1}{B} \sum_i \ell_i$ и определим $\Delta^{(\ell)} = \partial J_B / \partial Z^{(\ell)}$. Для бинарного выхода с BCE

$$\Delta^{(L)} = \frac{P - Y}{B} \in \mathbb{R}^{1 \times B};$$

для softmax с СЕ формула такая же, но $P, Y \in \mathbb{R}^{K \times B}$. Деление на B уже включено в $\Delta^{(L)}$ и распространяется дальше. Вторично усреднять градиенты параметров не нужно.

Для скрытых слоёв и параметров

$$\begin{aligned} \Delta^{(\ell)} &= (W^{(\ell+1)\top} \Delta^{(\ell+1)}) \odot \varphi'_\ell(Z^{(\ell)}), \\ \nabla_{W^{(\ell)}} J_B &= \Delta^{(\ell)} H^{(\ell-1)\top}, \quad \nabla_{b^{(\ell)}} J_B = \Delta^{(\ell)} \mathbf{1}_B. \end{aligned}$$

Первое произведение имеет размер $(d_\ell \times B)(B \times d_{\ell-1})$. Второе суммирует вклады столбцов. Среднее, а не сумма потерь, позволяет сохранять сопоставимый масштаб градиента при смене B ; при этом сам шум и поведение обучения всё равно меняются.

14.3 Эквивалентное соглашение

Можно определить столбцы $\tilde{\Delta}$ как производные отдельных ℓ_i , без деления на B . Тогда в градиентах весов и смещений появится явный множитель $1/B$. Обе записи верны. Ошибка — смешать их и получить лишнее либо пропущенное усреднение.

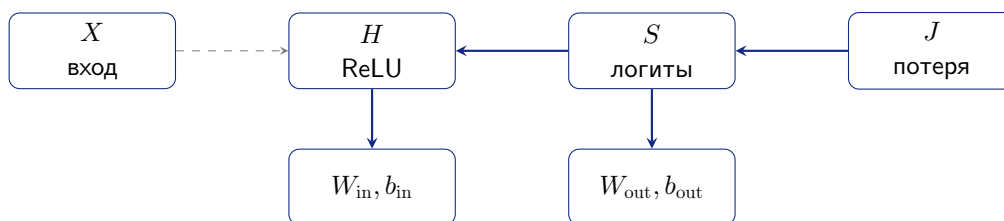
Формулы выше предполагают, что слои обрабатывают столбцы независимо. BatchNorm при обучении связывает объекты через статистики батча; для него обратный проход содержит дополнительные слагаемые. Градиент общего графа по-прежнему вычисляется правилом цепочки.

15 Автоматическое дифференцирование в PyTorch

Те же правила дифференцирования можно поручить программе. Во время прямого прохода PyTorch записывает связи между выполненными операциями. По этим связям затем проходит backward. Рассмотрим сеть с одним скрытым ReLU-слоем и одним выходным логитом для бинарной классификации.

В PyTorch объекты батча расположены строками: X имеет форму (B, d_{in}) , а бинарные цели y — $(B, 1)$. У Linear матрица весов имеет форму $(d_{\text{out}}, d_{\text{in}})$, поэтому операция над батчем записывается как $XW^T + b$. В формулах выше объекты были столбцами; разница только в расположении данных. Пусть X, y уже подготовлены, имеют вещественный тип и `requires_grad=False`, а $d_{\text{in}}, d_{\text{hidden}}$ задают ширины слоёв.

```
from torch import nn
model = nn.Sequential(
    nn.Linear(d_in, d_hidden), nn.ReLU(),
    nn.Linear(d_hidden, 1)
)
logits = model(X)
loss = nn.BCEWithLogitsLoss()(logits, y)
model.zero_grad(set_to_none=True)
loss.backward()
```



Синие стрелки показывают передачу чувствительности от потери к ранним операциям и параметрам. Пунктирная стрелка напоминает, что значения входа нужны для градиента первого слоя, даже если производная по самому входу не запрашивается. При обратном проходе Linear использует уже знакомые правила для аффинного преобразования, а ReLU — покомпонентную маску. Все градиенты относятся к параметрам, использованным в текущем прямом проходе.

15.1 Что хранит граф и куда попадают градиенты

У тензора, полученного дифференцируемой операцией, свойство `grad_fn` указывает на узел её обратного прохода. Связи с предыдущими узлами доступны через `next_functions`. Начиная с `loss.grad_fn`, можно проследить, как потеря зависит от логитов, скрытых активаций и параметров обоих Linear. Имена узлов зависят от реализации; для понимания графа важны операции и связи между ними.

Параметры сети — *листья* графа. Они не получены из других обучаемых тензоров в текущем прямом проходе и по умолчанию имеют `requires_grad=True`. После `loss.backward()` производные по ним появляются в поле `.grad`. Промежуточные активации тоже участвуют в дифференцировании, но в их поля `.grad` значения по умолчанию не записываются: для вычисления градиентов параметров это не нужно.

Входы X и метки y здесь не требуют градиентов. Поэтому от них нет ветвей к накопителям производных. Однако значение входа необходимо для градиента матрицы весов: в формуле $\nabla_W \ell = \delta h^\top$ участвует вход слоя h . PyTorch сохраняет нужные для backward тензоры независимо от того, нужен ли градиент по каждому из них. Отсутствие `X.grad` не означает, что X не используется при вычислении градиента W .

Вызов `backward()` вычисляет производные и складывает их с уже накопленными `.grad`. Он не обновляет веса. Поэтому перед обычным новым шагом градиенты очищают через `zero_grad`, а после полного backward выполняют `optimizer.step()`. Если очистку намеренно пропустить, несколько прямых и обратных проходов могут накопить сумму градиентов; масштаб этой суммы нужно учесть в алгоритме.

Граф строится заново при каждом прямом проходе. Поэтому Python может выбирать ветвь по условию или повторять блок в цикле: дифференцируются те операции, которые действительно были выполнены. Сохранённые для backward промежуточные значения после обычного обратного прохода освобождаются, если специально не запрошено сохранение графа.

Операции внутри `torch.no_grad()` не записываются в историю дифференцирования. Вызов `detach()` возвращает тензор, отсоединённый от прежней истории: вычисления с ним больше не передают градиент к предшествующим операциям. Это полезно, когда разрыв нужен по смыслу алгоритма. Если отсоединить скрытую активацию случайно, ранние слои перестанут получать через неё обучающий сигнал.

16 Проверка градиентов и поиск ошибок

Для небольшого тестового примера компоненту градиента можно проверить независимо:

$$\frac{\partial J}{\partial \theta_j} \approx \frac{J(\theta + \varepsilon e_j) - J(\theta - \varepsilon e_j)}{2\varepsilon}.$$

Слишком большое ε даёт ошибку приближения, слишком маленькое — потерю точности при вычитании близких чисел. Выбирают несколько разумных масштабов, используют повышенную точность и сравнивают как абсолютную, так и относительную ошибку. Вблизи излома ReLU правая и левая точки могут принадлежать разным гладким областям, поэтому такая проверка требует отдельно учитывать негладкость.

Для P параметров центральная разность требует около $2P$ прямых вычислений потери, тогда как backprop переиспользует промежуточные результаты и получает все компоненты одним обратным обходом. Поэтому конечные разности удобны для небольшой проверки, но слишком дороги как основной способ обучения большой сети.

Для Dropout при двух вычислениях нужно фиксировать одну маску либо отключить случайность. Для BatchNorm необходимо зафиксировать состав батча и контролировать обновление накопленных статистик. Иначе разность измеряет не только влияние θ_j .

16.1 Как искать ошибку в обучении

Начинают с маленького детерминированного примера: проверяют размеры, выходы, конечность потерь и градиентов. Затем пробуют сильно уменьшить потерю на нескольких объектах. Если простая достаточно выразительная сеть не может запомнить крошечную выборку без регуляризации, стоит проверить вычисления, метки и настройку шага. Для такой проверки временно убирают сильную регуляризацию и случайное выключение активаций. Проверять запоминание противоречивых меток бессмысленно: детерминированная функция не может дать два разных точных ответа на один и тот же вход. Успешное запоминание — проверка конвейера, а не доказательство качества на новых данных.

17 Глубина и устойчивость обучающего сигнала

Векторный backprop последовательно умножает градиент на транспонированные матрицы весов и производные активаций. Если локальный якобиан слоя равен A_ℓ , то при передаче через много слоёв возникает произведение

$$\nabla_{h^{(0)}} \ell = A_1^\top A_2^\top \cdots A_L^\top \nabla_{h^{(L)}} \ell.$$

Направления, которые систематически сжимаются, могут получить очень малый сигнал; направления, которые систематически растягиваются, — огромный. Это называют затуханием и взрывом градиента. Одних норм матриц недостаточно, чтобы точно предсказать эффект: важны также направления и их взаимное расположение.

Для сигмоиды локальная производная не больше $1/4$. Много таких множителей способны ослабить сигнал, хотя веса могут частично его усилить. ReLU устраняет этот конкретный множитель на положительной ветви, но не гарантирует устойчивость произведения весовых матриц и может занулять часть направлений.

17.1 Почему нельзя начать с одинаковых нейронов

Если нейроны одного скрытого слоя имеют одинаковые входящие веса и смещения, а их связи со следующим слоем также симметричны, то они вычисляют одинаковые выходы и получают одинаковые градиенты. После обновления равенство сохранится. Несколько нейронов будут повторять один и тот же признак вместо того, чтобы обучаться разным зависимостям.

Случайные веса нарушают эту симметрию. Нулевые смещения часто допустимы, если веса уже различаются. Но независимая случайность сама по себе ещё не решает вопрос масштаба: слишком большие веса могут быстро увеличить активации или отправить сигмоиду в насыщение, слишком маленькие — ослабить сигнал через многие слои.

Входные признаки масштабируют по статистикам train. После запуска обучения следят за величинами активаций и градиентов, а также за конечностью потерь. При редких чрезмерных градиентах используют ограничение нормы, которое рассмотрим при обучении рекуррентных сетей. Оно меняет шаг, но не исправляет неудачный масштаб всех слоёв или ошибку в вычислении потерь.

18 Масштаб инициализации: Xavier и He

Рассмотрим преактивацию $z_j = \sum_{k=1}^{d_{\text{in}}} W_{jk} h_k$ при нулевом смещении. Для грубой оценки предположим, что веса независимы, имеют нулевое среднее и дисперсию s^2 , не зависят от входов, а вторые моменты входных координат одинаковы. Тогда перекрёстные слагаемые в среднем исчезают:

$$\mathbb{E}[z_j^2] \approx d_{\text{in}} s^2 \mathbb{E}[h_k^2].$$

Чтобы линейный слой не менял этот масштаб, хотелось бы взять $s^2 \approx 1/d_{\text{in}}$. Аналогичная оценка для обратного прохода приводит к $1/d_{\text{out}}$. При разных ширинах оба требования нельзя точно выполнить одним числом.

Инициализация Xavier, или Glorot [8], использует компромисс

$$\text{Var}(W_{jk}) = \frac{2}{d_{\text{in}} + d_{\text{out}}}.$$

Его можно реализовать нормальным распределением с этой дисперсией либо равномерным на отрезке

$$\left[-\sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}, \sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}} \right].$$

Xavier согласует примерно линейный режим активации и сопоставимые масштабы прямого и обратного сигналов; для конкретной нелинейности вводят поправку. Такая инициализация не гарантирует отсутствия насыщения.

Для ReLU при симметричном распределении преактивации половина второго момента отсекается:

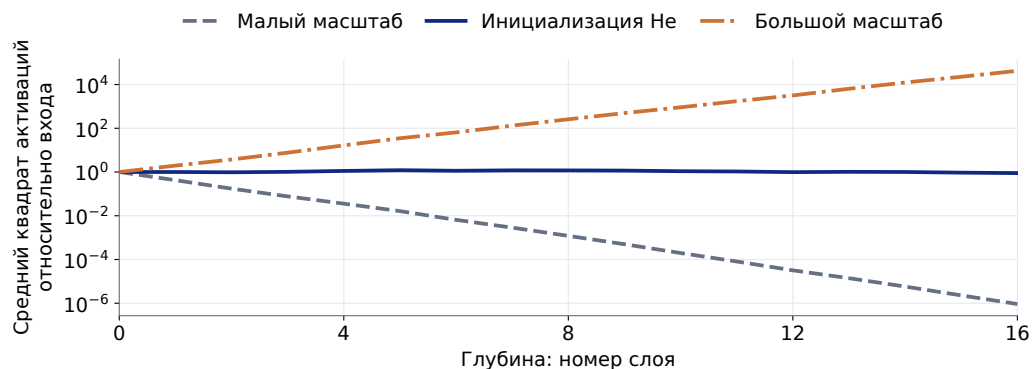
$$\mathbb{E}[\text{ReLU}(z)^2] = \frac{1}{2} \mathbb{E}[z^2].$$

Подставляя это в оценку масштаба, получаем инициализацию He [6] для прямого прохода:

$$\text{Var}(W_{jk}) \approx \frac{2}{d_{\text{in}}}.$$

He сохраняет *второй момент*: среднее ReLU обычно положительно. Используют центрированные нормальные веса; вариант для обратного сигнала заменяет d_{in} на d_{out} .

При обучении веса и активации становятся зависимыми. Инициализация задаёт исходный масштаб, но не гарантирует устойчивости дальнейших шагов.



Слишком малые веса гасят активации, слишком большие усиливают их от слоя к слою. На рисунке показан прямой проход случайных ReLU-сетей до обучения, без смещений; кривые усреднены по нескольким инициализациям. Масштаб He учитывает отсечение ReLU и помогает сохранять величину сигнала.

19 ResNet: учить поправку к представлению

Пусть вход и выход блока имеют одинаковый размер. Вместо прямого вычисления нового представления $H(h)$ запишем

$$h_{\text{next}} = h + F(h; \theta).$$

Функция F называется остаточной ветвью, а прямое сложение с h — shortcut, или остаточным соединением. Например,

$$F(h) = W_2 \text{ReLU}(W_1 h + b_1) + b_2.$$

Если полезное преобразование близко к тождественному, достаточно приблизить $F(h)$ к нулю. Блок сохраняет вход через прямую ветвь, а обучаемая часть ищет поправку к уже полезному представлению. Это основная идея остаточного обучения в работе Хе и соавторов [2]. В обычной последовательности слоёв для сохранения входа пришлось бы отдельно подобрать веса, реализующие почти тождественное преобразование.

Пусть от следующих слоёв пришёл $g = \partial \ell / \partial h_{\text{next}}$. У входа два пути влияния на выход: прямой и через F . Правило сложения ветвей даёт

$$\nabla_h \ell = (I + J_F(h))^T g = g + J_F(h)^T g.$$

Здесь J_F — якобиан остаточной ветви. Прямой вклад g не проходит через её внутренние веса и активации. При этом итоговый градиент может и сократиться из-за компенсации, например если $J_F \approx -I$. Остаточное соединение облегчает передачу информации и оптимизацию, но не гарантирует ненулевые и ограниченные градиенты при любых параметрах.

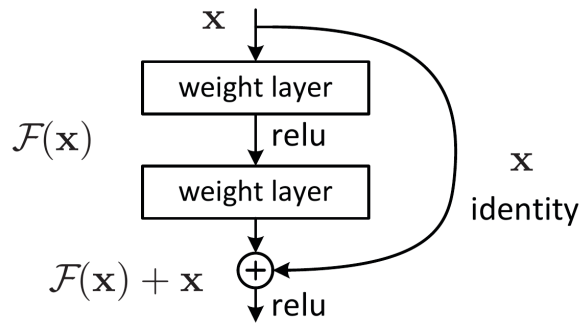
19.1 Размеры и активация после сложения

Складывать можно только согласованные формы. При переходе между разными ширинами используют проекцию P :

$$h_{\text{next}} = Ph + F(h), \quad \nabla_h \ell = (P + J_F)^\top g.$$

Теперь прямая ветвь уже не обязана быть тождественной. В свёрточной сети проекция может одновременно менять число каналов и пространственный размер.

В исходном остаточном блоке [2] после сложения стоит ReLU. На рисунке вход обозначен x :



Для такого блока в наших обозначениях

$$u = h + F(h), \quad h_{\text{next}} = \text{ReLU}(u), \quad \nabla_h \ell = (I + J_F)^\top (g \odot \mathbf{1}_{u>0}).$$

Дополнительная маска может занулить и прямой вклад. Формула $g + J_F^\top g$ относится к сумме до активации. Для других схем остаточного блока нужно учитывать дополнительные операции.

20 BatchNorm: нормировка внутренних признаков

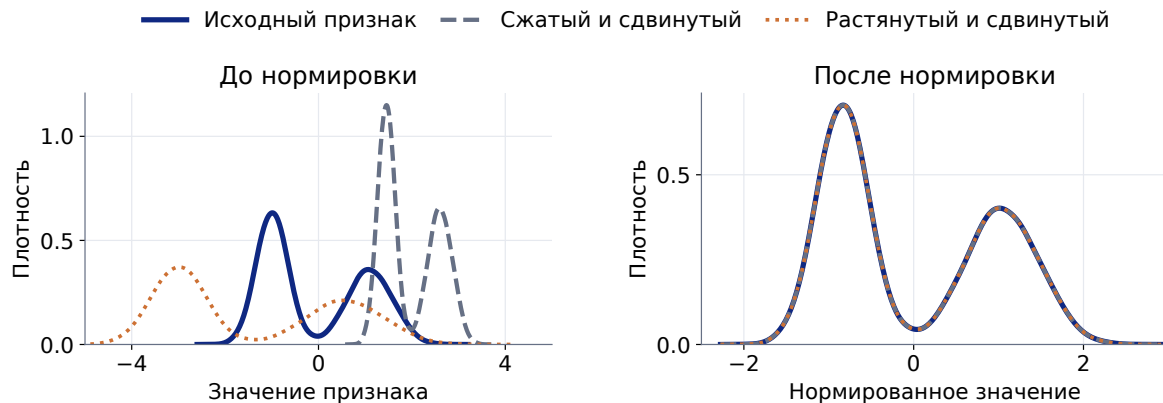
Рассмотрим матрицу предактиваций $Z \in \mathbb{R}^{d \times B}$. Во время обучения Batch Normalization, BN, отдельно для каждого признака j считает статистики по объектам:

$$\mu_j = \frac{1}{B} \sum_{i=1}^B z_{ji}, \quad v_j = \frac{1}{B} \sum_{i=1}^B (z_{ji} - \mu_j)^2,$$

$$\hat{z}_{ji} = \frac{z_{ji} - \mu_j}{\sqrt{v_j + \varepsilon}}, \quad a_{ji} = \gamma_j \hat{z}_{ji} + \beta_j.$$

Величина $\varepsilon > 0$ предотвращает деление на ноль и влияет на масштаб при малой дисперсии. γ_j и β_j — обучаемые параметры масштаба и сдвига. Средние и дисперсии — вычисляемые статистики, а не параметры, обновляемые градиентным спуском.

Нормировка не запрещает сети использовать другой средний уровень и масштаб: для этого есть γ, β . Также дисперсия $\hat{z}$ равна $v/(v + \varepsilon)$, а не в точности 1 при ненулевом ε . После обучаемого преобразования среднее равно β , а дисперсия дополнительно умножается на γ^2 .



Пунктирные кривые — сжатая и растянутая копии исходного признака на тех же объектах; каждая также сдвинута. Это разные масштабы и средние уровни одной зависимости. Вычитание среднего убирает сдвиг. При растяжении одновременно увеличиваются отклонения от среднего и стандартное отклонение, на которое их делят. Поэтому после нормировки значения совпадают, если пренебречь ε ; при ненулевом ε совпадение приближённое. Форма распределения при этом сохраняется: нормировка не превращает произвольное распределение в нормальное.

Во время обучения каждый нормированный выход зависит от остальных объектов через μ и v . Если заменить один объект, могут измениться выходы его соседей по батчу. Возникающий шум иногда дополнительно регуляризует сеть, но для слишком маленьких или зависимых батчей неточность статистик способна мешать.

Частая последовательность в полносвязном блоке — аффинный слой, BN, активация. В свёрточных сетях для тензора (B, C, H, W) статистики собирают по осям B, H, W отдельно для каждого канала. Обучаемые масштаб и сдвиг общие для пространственных позиций канала. Соседние пиксели зависимы, поэтому большое число позиций само по себе не гарантирует качественных статистик. Нормировка по признакам одного объекта использует другие оси и относится к другому методу.

20.1 Разделение направления весов и масштаба признака

Рассмотрим один нейрон перед BN на фиксированном обучающем батче. Разложим его ненулевой вектор весов на длину и направление:

$$w = ru, \quad r = \|w\| > 0, \quad \|u\| = 1, \quad z_i = w^\top x_i + b = ru^\top x_i + b.$$

Направление u определяет, какую линейную комбинацию входов выделяет нейрон. Длина r растягивает все значения этой комбинации. Обозначим

$$\bar{x} = \frac{1}{B} \sum_i x_i, \quad s_B^2(u) = \frac{1}{B} \sum_i [u^\top (x_i - \bar{x})]^2.$$

Тогда среднее преактиваций равно $\mu_z = ru^\top \bar{x} + b$, а дисперсия равна $v_z = r^2 s_B^2(u)$. Смещение b вычитается вместе со средним. Подставим эти выражения в BN:

$$\hat{z}_i = \frac{ru^\top (x_i - \bar{x})}{\sqrt{r^2 s_B^2(u) + \varepsilon}} = \frac{u^\top (x_i - \bar{x})}{\sqrt{s_B^2(u) + \varepsilon/r^2}}.$$

При $\varepsilon = 0$ и $s_B^2(u) > 0$ длина r полностью исчезает из нормированного ответа:

$$a_i = \gamma \frac{u^\top (x_i - \bar{x})}{s_B(u)} + \beta.$$

Направление весов выбирает признак, γ задаёт масштаб и знак его выхода, а β — средний уровень. Это и есть разделение, или *decoupling*, направления весов и масштаба выходного признака. При положительном ε независимость от r приближённая: она точнее, когда $\varepsilon \ll r^2 s_B^2(u)$. BN не заставляет $\|w\|$ быть равной единице и не нормирует сами веса: нормируются преактивации с помощью статистик батча [3].

20.2 Почему длина весов всё ещё влияет на обучение

Даже если масштабирование весов не меняет выход сети, оно может менять ход обучения. В идеальном случае масштабно-инвариантная потеря удовлетворяет $J(cw) = J(w)$ при $c > 0$. Её градиенты связаны соотношениями

$$\nabla J(cw) = \frac{1}{c} \nabla J(w), \quad w^\top \nabla J(w) = 0.$$

Следовательно, увеличение длины весов уменьшает градиент по ним. Для обычного SGD без отдельного штрафа на веса эффективный масштаб изменения направления пропорционален η/r^2 [9]. Норма может меняться в процессе обучения, хотя прямой ответ от неё не зависит. С ненулевым ε , регуляризацией или адаптивным оптимизатором простое описание требует поправок.

BN убирает часть зависимости выхода от масштаба предыдущих весов, а γ, β позволяют настраивать выход отдельно. Это может улучшать геометрию задачи и позволять больший learning rate. Santurkar и соавторы [5] связывают пользу BN с более гладким поведением потерь и градиентов. Одного объяснения через уменьшение internal covariate shift для этого недостаточно.

20.3 BatchNorm при применении и в обратном проходе

На новых данных нельзя полагаться на случайный состав текущего батча: один и тот же объект должен получать воспроизводимый ответ. При обучении поэтому накапливают оценки среднего и дисперсии, например экспоненциальным усреднением:

$$\bar{\mu} \leftarrow (1 - \alpha)\bar{\mu} + \alpha\mu_B, \quad \bar{v} \leftarrow (1 - \alpha)\bar{v} + \alpha v_B.$$

При применении используют фиксированные $\bar{\mu}, \bar{v}$ и обученные γ, β . Детали оценки дисперсии и определения коэффициента усреднения зависят от реализации; важно различать деление на B при нормировке текущего батча и возможную поправку для накопленной статистики.

Разделение направления и масштаба выше опиралось на пересчёт статистик текущего батча. В режиме применения простое умножение w на положительное число обычно меняет ответ: накопленные $\bar{\mu}, \bar{v}$ остаются прежними. Поэтому свойство обучающего BN нельзя без оговорок переносить на режим `eval`.

Для одного признака обозначим $g_i = \partial\ell/\partial a_i$ и $\rho = \sqrt{v + \varepsilon}$. Обратный проход через обучающий BN имеет вид

$$\begin{aligned} \frac{\partial\ell}{\partial\beta} &= \sum_i g_i, & \frac{\partial\ell}{\partial\gamma} &= \sum_i g_i \hat{z}_i, \\ \frac{\partial\ell}{\partial z_i} &= \frac{\gamma}{\rho} \left[g_i - \frac{1}{B} \sum_k g_k - \hat{z}_i \frac{1}{B} \sum_k g_k \hat{z}_k \right]. \end{aligned}$$

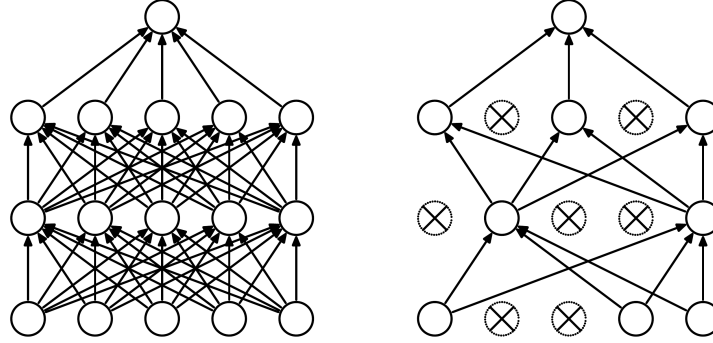
Два средних в скобках учитывают зависимости через статистики батча. Если считать μ и v константами, они пропадут и получится другая производная. Формула уже использует входящий градиент с той нормировкой потери, которая принята в остальной сети.

20.4 Что BN улучшает и где возникают ограничения

Эффект BatchNorm зависит от архитектуры и данных. Маленькие или сильно зависимые батчи дают шумные статистики. В полносвязном случае при $B = 1$ имеем $z - \mu = 0$, и такой обучающий BN теряет информацию входа. Если при применении забыть включить режим оценки, ответ может зависеть от соседних объектов, а статистики могут нежелательно обновляться. Сдвиг распределения между `train` и применением также снижает пригодность накопленных оценок.

21 Dropout: регуляризация случайными масками

Dropout — способ регуляризации, при котором во время обучения случайно обнуляют часть активаций скрытого слоя. Нейрон, выключенный на текущем проходе, снова может участвовать в следующем. Веса при этом не удаляются.



Слева показана полная сеть, справа — одна из случайных масок; выключенные нейроны перечёркнуты [4]. Для каждой координаты независимо выбирают маску

$$m_j \sim \text{Bernoulli}(q), \quad q = 1 - p_{\text{drop}}, \quad 0 < q \leq 1.$$

Здесь p_{drop} — вероятность выключения, q — вероятность сохранения. В распространённом *inverted dropout* вычисляют

$$\tilde{h}_j = \frac{m_j}{q} h_j.$$

При обучении случайную маску выбирают заново. Способ её применения к разным активациям зависит от разновидности слоя. Выключенные координаты становятся нулями, сохранённые увеличиваются в $1/q$ раз.

При фиксированном h математическое ожидание отдельной активации сохраняется:

$$\mathbb{E}_m[\tilde{h}_j | h] = h_j, \quad \text{Var}_m(\tilde{h}_j | h) = h_j^2 \frac{1-q}{q}.$$

Чем больше вероятность выключения, тем сильнее возмущение. Масштаб $1/q$ сохраняет среднюю величину активации, но отдельный проход остаётся случайным.

21.1 Применение и backward

В режиме применения *inverted dropout* выключен: $\tilde{h} = h$, без случайной маски и без дополнительного масштабирования. Масштабирование уже выполнялось при обучении. В другой, исторически распространённой записи масштабируют веса при применении; смешивать два соглашения нельзя.

Маска не является обучаемым параметром. Для фиксированной маски backward прост:

$$\frac{\partial \ell}{\partial h_j} = \frac{m_j}{q} \frac{\partial \ell}{\partial \tilde{h}_j}.$$

Выключенная координата не получает градиент через этот путь на данном шаге. На следующем шаге другая маска может её сохранить.

21.2 Почему Dropout не равен точному усреднению сетей

Обучение с Dropout минимизирует среднюю потерю по случайным маскам:

$$J_{\text{drop}}(\theta) = \frac{1}{n} \sum_{i=1}^n \mathbb{E}_m [\ell(f_{\theta}(x_i; m), y_i)].$$

На каждом проходе случайная маска даёт оценку этой цели. Dropout можно интерпретировать как обучение многих случайно прореженных сетей с общими весами [4]. Но равенство ожиданий отдельных активаций не означает, что детерминированный выход всей нелинейной сети в точности равен среднему по маскам:

$$\mathbb{E}_m[f(\tilde{h})] \neq f(\mathbb{E}_m[\tilde{h}])$$

в общем случае: нелинейную функцию нельзя свободно переставить с математическим ожиданием. Детерминированное применение поэтому не является точным усреднением всех прореженных сетей.

На одном шаге часть полезных активаций недоступна, на другом недоступна другая часть. Следующие слои обучаются делать предсказание при разных наборах доступных активаций. Это ослабляет зависимость от одного конкретного признака или сочетания нейронов и может улучшать обобщение. Так Dropout ограничивает чрезмерную подгонку train, не уменьшая постоянно число нейронов. Слишком сильное выключение, напротив, затрудняет подгонку даже обучающей выборки. Его уровень выбирают по валидации, а не по предположению «чем больше регуляризация, тем лучше».

22 Свёрточная сеть и пространственная структура

В изображении важны соседство пикселей и повторение похожих узоров в разных местах. Свёрточный слой использует эту структуру: обрабатывает небольшие окрестности одним и тем же набором весов.

Таблица этих весов называется *ядром*, или фильтром. Она перемещается по изображению и в каждой позиции вычисляет взвешенную сумму пикселей. Полученная *карта признака* показывает отклик фильтра в разных местах.

Для одного входного канала, ядра $k \times k$, единичного шага и без дополнения краёв:

$$Y_{ij} = b + \sum_{a=0}^{k-1} \sum_{d=0}^{k-1} K_{ad} X_{i+a, j+d}.$$

В библиотеках CNN ядро обычно не переворачивают: это *взаимная корреляция*. Математическая свёртка использовала бы перевёрнутое ядро $K_{k-1-a, k-1-d}$.

Совместное использование весов позволяет искать один признак во всех позициях. Поэтому число параметров слоя не растёт с размером изображения, хотя вычислений и памяти для карт становится больше.

23 Свёртка как локальная обработка сигнала

Изображение можно рассматривать как пространственный сигнал яркости $I(x, y)$. Локальные фильтры сглаживают его, выделяют изменения или усиливают детали.

Усреднение. Если

$$K_{ad} \geq 0, \quad \sum_{a,d} K_{ad} = 1,$$

фильтр вычисляет взвешенное среднее по окрестности и сохраняет постоянный фон. Независимый случайный шум частично компенсируется, но края и тонкие линии тоже размываются. Гауссов фильтр придаёт ближайшим пикселям больший вес. Размытие в линейной, одинаковой по кадру модели оптики тоже описывается свёрткой.

Производная. Разность соседних яркостей оценивает изменение сигнала:

$$\frac{I(x + \Delta x, y) - I(x - \Delta x, y)}{2\Delta x} \approx \frac{\partial I}{\partial x}.$$

Знак показывает направление перехода между тёмным и светлым, модуль — силу перепада. Граница может возникнуть из-за контура предмета, тени или рисунка на поверхности.

23.1 Sobel, повышение резкости и обучаемые фильтры

Sobel сочетает разность по одной оси с усреднением по другой. Отклик G_x выделяет вертикальные границы, G_y — горизонтальные. Величина $\sqrt{G_x^2 + G_y^2}$ объединяет силу перепада по обеим осям; её шкала зависит от нормировки ядра.

Повышение резкости. К изображению добавляют усиленную разность между ним и сглаженной версией $S(I)$:

$$I_{\text{sharp}} = I + \lambda(I - S(I)), \quad \lambda > 0.$$

Другой вариант использует дискретный лапласиан:

$$I_{\text{sharp}} = I - \lambda \Delta I.$$

Обе операции подчёркивают локальные различия и могут усиливать шум. Утраченные детали от этого не восстанавливаются.

В CNN веса фильтров обучаются по конечной задаче. Отклик $z = \langle K, \text{окно} \rangle + b$ зависит от локального узора и масштаба входных значений. Смещение задаёт порог, ReLU оставляет положительную часть, а другой фильтр может выделить противоположный перепад. Следующие слои объединяют отклики в более сложные признаки.

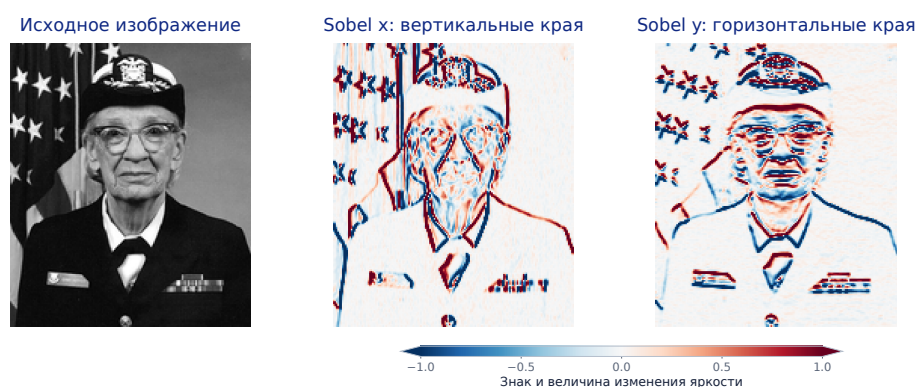
23.2 Действие фильтров на изображении

На фотографии усреднение размывает мелкие детали, а усиление резкости подчёркивает их.

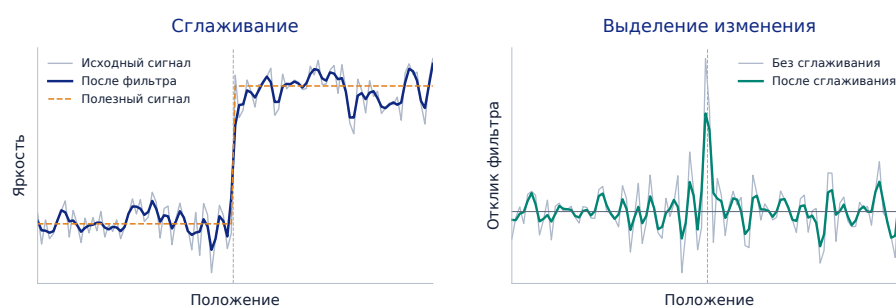


Одна обработка ядром 3×3 ; одинаковая шкала яркости $[0, 1]$

Карты Sobel показывают перепады по разным направлениям и сохраняют знак отклика. При переходе к модулю градиента знак теряется.



На профиле яркости усреднение сглаживает ступеньку, а разностный фильтр локализует переход. Масштаб отклика зависит от нормировки ядра.



24 Размеры свёрточного слоя

Три настройки управляют тем, где расположены окна и какие пиксели они используют. *Stride* s — шаг между соседними положениями окна. *Padding* p — число добавленных позиций с каждой стороны. *Dilation* d — расстояние между соседними отсчётами ядра; $d = 1$ означает обычное плотное ядро.

Для входа размера n вдоль одной оси и ядра с k весами эффективный размер окна равен

$$k_{\text{eff}} = d(k - 1) + 1.$$

При симметричном дополнении p и обычном valid-расположении окон выходной размер равен

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - d(k - 1) - 1}{s} \right\rfloor + 1.$$

Для высоты и ширины формулу применяют отдельно.

Почему появляется целая часть? Начало окна должно находиться достаточно далеко от конца, чтобы всё эффективное окно поместилось. Если после последнего полного шага осталось меньше s позиций, дополнительного выхода не возникает. Для несимметричного дополнения вместо $2p$ используется сумма левого и правого дополнений.

Смысл настроек. Большой stride уменьшает пространственное разрешение и вычисления, но часть сдвигов и мелких деталей может стать неразличима. Подходящий padding помогает сохранить размер выходной карты и включить краевые пиксели в большее число окон, однако добавленные значения являются предположением о том, что находится за границей кадра. Нулевое дополнение само способно создать искусственный перепад возле светлого края.

Dilation позволяет смотреть на более широкую область при том же числе весов. Между используемыми позициями появляются промежутки: некоторые точки внутри охвата пропускаются. Увеличенный охват не равен столь же плотному анализу всей окрестности.

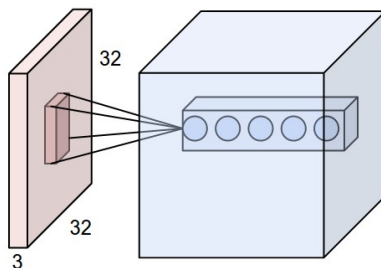
25 Каналы и параметры свёртки

RGB-изображение имеет три входных канала: красный, зелёный и синий. Чтобы получить *одну* выходную карту, стандартная свёртка использует отдельную таблицу весов для каждого входного канала, затем складывает результаты:

$$Y_{o,i,j} = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{a=0}^{k_h-1} \sum_{d=0}^{k_w-1} K_{o,c,a,d} X_{c,i+a,j+d}.$$

Здесь для простоты stride 1 и padding 0. Индекс o выбирает выходной канал. Один фильтр имеет форму $C_{\text{in}} \times k_h \times k_w$, а весь набор весов — $C_{\text{out}} \times C_{\text{in}} \times k_h \times k_w$. Эти формы соответствуют обычной свёртке без групп.

Например, один фильтр может сравнивать яркость справа и слева одновременно в трёх цветах. Другой может реагировать на красноватый участок на зелёном фоне. Уже после первого слоя каналы обычно не соответствуют отдельным физическим цветам: это обученные карты признаков.



Одна RGB-окрестность подаётся нескольким выходным фильтрам; каждый охватывает все три входных канала. Схема Stanford CS231n.

Число параметров. У каждого выходного канала свой набор входных весов и одно смещение:

$$N_{\text{conv}} = C_{\text{out}}(C_{\text{in}}k_hk_w + 1).$$

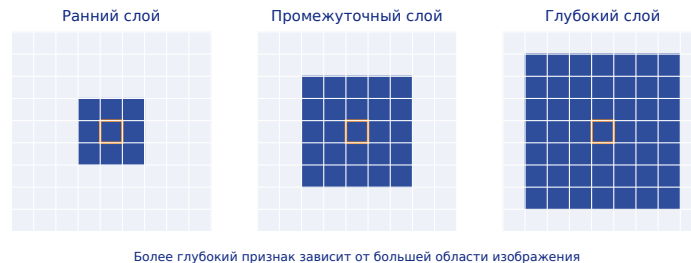
Одно и то же смещение используется во всех пространственных позициях выходной карты.

Свёртка 1×1 . Она не смешивает соседние позиции, но в каждой позиции вычисляет линейные комбинации всех каналов. Так можно менять число каналов в остаточной проекции. При увеличенном stride эта же операция выбирает более редкую пространственную сетку.

Обратный проход. Один вес участвует в вычислениях многих пространственных позиций, поэтому его градиент складывает вклады из всех этих позиций и всех объектов батча. Совместное использование означает именно общий параметр, а не независимые копии, которые лишь одинаково инициализировали.

26 Поле восприятия, pooling и сдвиги

Каждый элемент карты зависит от некоторой области исходного изображения. Это его *поле восприятия*. С каждым следующим локальным слоем охват увеличивается: его входы уже зависят от своих окрестностей. Между слоями есть нелинейности, поэтому стек не эквивалентен одному линейному ядру такого размера.



Чтобы учитывать изменение шага сетки, обозначим r_ℓ размер теоретического поля восприятия, j_ℓ — расстояние между соседними центрами выхода в координатах исходного изображения. При $r_0 = j_0 = 1$:

$$r_\ell = r_{\ell-1} + (k_\ell - 1)d_\ell j_{\ell-1}, \quad j_\ell = s_\ell j_{\ell-1}.$$

На краях часть этого формального охвата может приходиться на padding. Реальный вклад разных пикселей внутри охвата зависит от весов и нелинейностей.

Max pooling. Вместо обучаемой взвешенной суммы берём максимум в локальном окне. У операции нет обучаемых параметров. Она сохраняет сильнейший локальный отклик; при увеличенном шаге окна пространственное разрешение уменьшается. Точное положение максимума внутри окна при этом теряется. Когда сильный отклик переходит в соседнее окно, результат может резко измениться, поэтому max pooling не обеспечивает устойчивости ко всем сдвигам.

Average pooling. В локальном окне вместо максимума можно брать среднее. Тогда ответ характеризует общий уровень активации, а не только самый сильный отклик. Слабые значения участвуют наравне с сильными, поэтому локальный пик обычно становится менее выраженным. В обратном проходе среднее распределяет чувствительность между элементами окна; максимум передаёт её выбранному максимальному элементу, с отдельным соглашением при равенстве максимумов.

Global average pooling, GAP. Усредним каждую карту целиком:

$$g_c = \frac{1}{HW} \sum_{i,j} a_{cij}.$$

Из $H \times W \times C$ получается вектор длины C . Он описывает среднюю выраженность каждого признака и позволяет обойтись без большого полносвязного слоя над всеми пикселями. Но расположение признаков существенно теряется, что не всегда допустимо, например при локализации объекта.

Эквивариантность и инвариантность. Для идеальной свёртки со stride 1 сдвиг входа сдвигает карту: $f(Tx) = Tf(x)$. Это эквивариантность. Инвариантность означает неизменный ответ $f(Tx) = f(x)$. Усреднение пространственных карт может приближать такой эффект в классификации, но конечные края, padding, stride и обрезание входа ограничивают точное равенство.

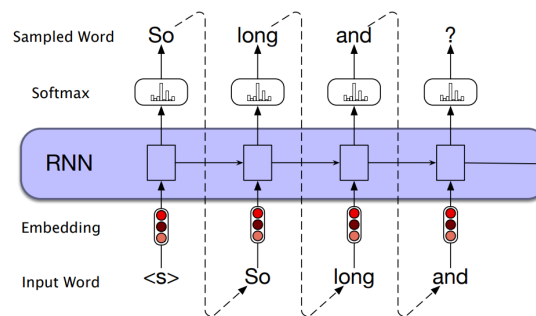
27 Рекуррентная сеть: перенос состояния

Элементом последовательности может быть любой объект, представленный вектором: слово, фрагмент звукового сигнала, измерение или кадр видео. Для таких данных важен порядок. Далее будем рассматривать последовательности слов. Например, “не очень хороший” и “очень хороший” различаются по смыслу, хотя обе фразы содержат слово “хороший”.

Рекуррентная сеть, RNN, читает последовательность $x_1, \dots, x_T$ и после каждого шага обновляет скрытое состояние h_t . Оно является обучаемым представлением уже увиденного префикса. Простая ячейка имеет вид

$$h_t = \tanh(Wx_t + Uh_{t-1} + b).$$

При $x_t \in \mathbb{R}^d$, $h_t \in \mathbb{R}^m$ имеем $W \in \mathbb{R}^{m \times d}$, $U \in \mathbb{R}^{m \times m}$, $b \in \mathbb{R}^m$. Начальное h_0 обычно задают нулями либо обучают. Все шаги используют *одни и те же* W, U, b .



Рекуррентная генерация. Daniel Jurafsky, James H. Martin, *Speech and Language Processing*.

Петля на компактной схеме означает перенос значения на следующий шаг, а не бесконечное вычисление внутри одного шага. После развёртки перед нами обычный ориентированный граф длины T . Число применений ячейки растёт с длиной последовательности; число её параметров остаётся $md + m^2 + m$.

Как подать слова? Сначала текст разбивают на токены и каждому сопоставляют индекс словаря. По индексу w_t берут строку обучаемой таблицы вложений $E \in \mathbb{R}^{V \times d}$:

$$x_t = E[w_t].$$

Такой вектор называют embedding. Идентификатор токена — адрес строки, а не количественный признак: арифметические отношения между номерами не задают смысловых отношений между токенами. Таблица вложений обучается вместе с остальными весами. Временной ряд уже может давать числовой вектор x_t напрямую.

RNN может возвращать один ответ по последнему состоянию, например класс текста, или ответ на каждом шаге, например метку части речи. Для языковой модели состояние превращается в распределение следующего токена. В однонаправленной RNN h_t зависит только от $x_1, \dots, x_t$; будущие токены недоступны.

28 Языковая модель: обучение и генерация

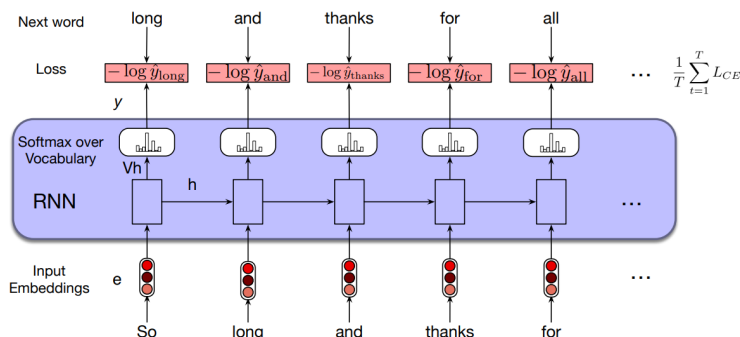
После чтения w_t вычислим логиты по словарю и вероятности следующего токена:

$$z_t = V_{\text{out}} h_t + b_{\text{out}}, \quad p_t = \text{softmax}(z_t).$$

В обучающем тексте правильный следующий токен w_{t+1} известен. Используем уже введённую кросс-энтропию для одного правильного класса, теперь по словарю:

$$\ell_t = -\log p_t[w_{t+1}], \quad L = \frac{1}{T-1} \sum_{t=1}^{T-1} \ell_t.$$

Чем меньшую вероятность модель назначает правильному токenu, тем больше потеря. Максимизация вероятности правильного продолжения обучает и выходной слой, и ячейку RNN, и embeddings.



Обучение следующему токenu. Daniel Jurafsky, James H. Martin, *Speech and Language Processing*.

Teacher forcing. При обучении на следующем шаге подаём настоящий токен из текста, даже если модель предсказала другой. Для фразы “кот спит дома” вход “кот” имеет цель “спит”, вход “спит” — цель “дома”. Состояние проходит через всю цепочку; цели сдвинуты на одну позицию.

Генерация. Сначала читаем стартовый токен или пользовательский префикс. Затем выбираем токен из распределения модели, подаём именно его на следующий шаг и повторяем. Можно выбирать самый вероятный токен либо случайно выбирать по распределению; это разные правила декодирования. На практике заканчиваем по специальному токenu конца или ограничению длины.

Ошибка при генерации меняет будущие входы. При teacher forcing модель обычно видит правильную историю, а при генерации — собственную. Это различие режимов помогает объяснить накопление ошибок: хорошая средняя потеря на правильных префиксах ещё не гарантирует безошибочного длинного продолжения.

29 Обратное распространение по времени

Обратное распространение по развёрнутой RNN называется *backpropagation through time*, ВРТТ. Градиент поздней потери проходит через промежуточные состояния; градиент общего параметра складывается из вкладов всех шагов.

Для простой RNN производная перехода равна

$$J_t = \frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(1 - h_t^2)U.$$

За r шагов возникает произведение:

$$\frac{\partial h_t}{\partial h_{t-r}} = J_t J_{t-1} \cdots J_{t-r+1}.$$

Повторяющееся сжатие ослабляет обучающий сигнал, усиление вызывает взрыв градиентов. В насыщении $\tanh$ множители $1 - h_t^2$ малы, что затрудняет обучение длинным зависимостям.

Gradient clipping. Норму ненулевого градиента ограничивают порогом $\tau > 0$:

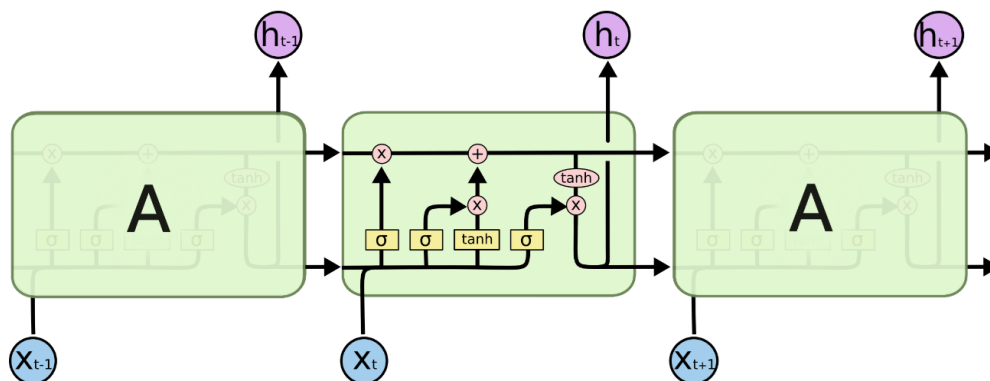
$$g \leftarrow g \min \left(1, \frac{\tau}{\|g\|} \right).$$

Нулевой градиент оставляют нулевым. Clipping сохраняет направление и уменьшает большие градиенты; затухание он не устраняет.

Усечённый ВРТТ. Длинную запись обрабатывают блоками по K шагов. Между блоками переносят числовое состояние, а граф разрывают через `detach()`. Производная через границу не проходит. Это экономит память и ограничивает горизонт обучения.

30 LSTM: управляемая память

В простой RNN каждый шаг полностью пересчитывает h_t через нелинейность. LSTM вводит отдельный вектор памяти c_t и управляемые операции над ним. Сохраняются два состояния: *cell state* c_t и *hidden state* h_t . Оба переходят к следующему шагу, но выполняют разные роли.



Christopher Olah, *Understanding LSTM Networks*. На схеме память обозначена C_t ; далее используем c_t .

Верхняя горизонтальная линия переносит память. Её значение можно умножить на коэффициент сохранения и дополнить новой записью. Нижняя линия передаёт h_t — выходное представление шага, которое используется для прогноза и для вычисления следующих управляющих коэффициентов.

Как читать рисунок. Каждая стрелка переносит целый вектор. Прямоугольники с σ и $\tanh$ — обучаемые преобразования с активациями. Кружки $\times$ означают покомпонентное умножение, $+$ — покомпонентную сумму. Соединение линий у входа прямоугольника означает конкатенацию, а разветвление — передачу копии значения в несколько операций.

Для краткости соберём доступную на шаге информацию в один вектор:

$$a_t = [h_{t-1}; x_t] \in \mathbb{R}^{m+d}.$$

Из него разные обучаемые слои получают коэффициенты $f_t, i_t, o_t \in (0, 1)^m$ и кандидата $\tilde{c}_t \in (-1, 1)^m$. “Вентиль” — это сигмоида и последующее умножение: коэффициент возле 0 почти закрывает путь, возле 1 почти открывает. Это мягкое числовое управление, а не программный оператор `if`.

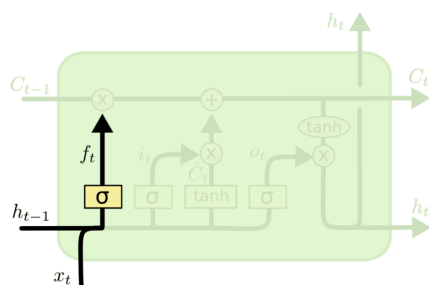
Пример задачи. Во фразе “Книги, которые лежали рядом с окном, оказались мокрыми” полезно сохранить сведения о множественном числе, несмотря на промежуточные слова. Это иллюстрация того, *какая* информация может быть полезна. Сведения о грамматическом числе могут быть распределены между несколькими координатами состояния; отдельная координата не обязана отвечать только за этот признак.

31 Вентили и обновление памяти LSTM

Сохранение старого состояния. Forget gate вычисляет

$$f_t = \sigma(W_f a_t + b_f).$$

Каждая координата f_t задаёт долю соответствующей координаты c_{t-1} , проходящую в прямую ветвь. Коэффициент близко к единице почти сохраняет старый вклад; коэффициент близко к нулю почти удаляет его. Вентили зависят от текущего входа и прошлого h , поэтому одна и та же сеть может менять режим по содержанию последовательности.



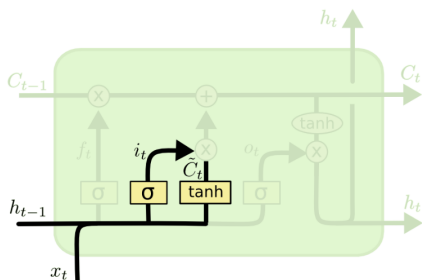
$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Ветвь сохранения. Christopher Olah, *Understanding LSTM Networks*.

Подготовка записи. Input gate и кандидат вычисляются независимо обучаемыми преобразованиями:

$$i_t = \sigma(W_i a_t + b_i), \quad \tilde{c}_t = \tanh(W_c a_t + b_c).$$

Кандидат содержит возможную поправку со знаком, а вентиль выбирает, какую её долю добавить. Благодаря $\tanh$ кандидат может быть и отрицательным: “записать” не означает обязательно увеличить координату памяти.



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

Вентиль записи и кандидат. Christopher Olah, *Understanding LSTM Networks*.

В обычной рассматриваемой LSTM f_t и i_t независимы: они не обязаны суммироваться в единицу. Можно почти полностью сохранить старое и одновременно добавить новое. Можно почти всё забыть и почти ничего не записать. Варианты со связанными вентилями существуют, но имеют другую параметризацию.

Все матрицы W_f, W_i, W_c имеют размер $m \times (m + d)$, а смещения — длину m . Размеры остальных вентиляей устроены так же: каждое преобразование получает конкатенацию входа и предыдущего скрытого состояния.

31.1 Обновление памяти и выход

Обновление памяти. Объединим сохранение и запись:

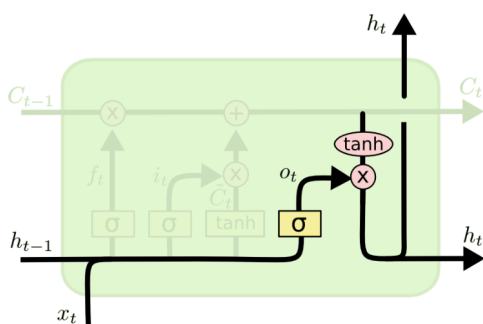
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t.$$

Это покоординатное сложение старого вклада и новой поправки. В отличие от простой RNN, весь c_t не пропускается через $\tanh$ на этой прямой линии. При последовательном накоплении его модуль может быть больше 1.

Выходное представление.

$$o_t = \sigma(W_o a_t + b_o), \quad h_t = o_t \odot \tanh(c_t).$$

Здесь $\tanh(c_t)$ ограничивает видимый диапазон, а output gate регулирует, что передать наружу. Если o_t мал, состояние c_t может сохраняться, хотя текущий h_t почти его не показывает.



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(c_t)$$

Выходной вентиль. Christopher Olah, *Understanding LSTM Networks*.

Обучаемые преобразования. Все вентили и кандидат имеют отдельные параметры. Поэтому при одинаковой ширине LSTM требует больше параметров и вычислений, чем простая RNN. Это плата за дополнительное управление памятью; полезность такого усложнения оценивают по задаче.

32 Обучение LSTM и работа с последовательностями

В формуле обновления памяти есть прямая ветвь от c_{t-1} к c_t . Если при анализе именно этой ветви фиксировать значения вентилей и кандидата, то

$$\left. \frac{\partial c_t}{\partial c_{t-1}} \right|_{f_t, i_t, \tilde{c}_t \text{ fixed}} = \text{diag}(f_t).$$

Через несколько шагов прямой покоординатный вклад содержит произведение коэффициентов сохранения. Если сеть научилась держать соответствующий клапан близким к 1, этот путь может переносить обучающий сигнал намного дальше.

Это не полный якобиан всей рекуррентной системы. Клапаны зависят от h_{t-1} , а оно связано с c_{t-1} ; существуют дополнительные пути влияния через них. Для полного состояния (c, h) производная имеет блочную структуру. Поэтому утверждение “градиент LSTM всегда равен f_t и никогда не затухает” неверно. Насыщение, длинные произведения и неудачные параметры остаются возможными; clipping тоже может быть полезен.

Связь с ResNet. И там, и здесь есть путь добавления к уже существующему представлению. В ResNet обычно складывают $x + F(x)$, а в LSTM старое состояние дополнительно умножают на обученный клапан. Сходство помогает понять роль аддитивного пути, но формулы и назначение архитектур различаются.

Границы последовательностей. Для независимых текстов или временных рядов состояния нужно сбрасывать. Иначе при обработке нового текста сеть будет использовать состояние, оставшееся от предыдущего независимого текста. Если читаем одну непрерывную запись блоками, перенос состояния допустим при сохранении порядка; при усечённом BPTT отдельно решаем, где отсоединять граф.

Разная длина. Последовательности в батче часто дополняют до общей длины специальным PAD. Одной маски потерь недостаточно, если затем использовать состояние после всех PAD: оно может измениться на искусственных шагах. Нужно остановить обновление для законченных последовательностей, использовать упаковку по длинам либо выбрать последнее настоящее состояние. В потерях PAD тоже исключают. Нулевой вектор входа сам по себе не замораживает LSTM.

Другие ячейки. GRU использует клапаны и единое состояние без отдельной линии c_t . Она часто требует меньше параметров, но превосходство над LSTM или простой RNN зависит от задачи, данных и обучения. Двухнаправленная модель читает контекст в обе стороны и полезна там, где вся последовательность уже доступна. Для причинной генерации будущие токены так использовать нельзя.

Литература

- [1] Deep Learning: Deep Feedforward Networks.
- [2] Deep Residual Learning for Image Recognition.
- [3] Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift.
- [4] Dropout: A Simple Way to Prevent Neural Networks from Overfitting.
- [5] How Does Batch Normalization Help Optimization?
- [6] Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification.
- [7] Adam: A Method for Stochastic Optimization.
- [8] Understanding the Difficulty of Training Deep Feedforward Neural Networks.
- [9] Theoretical Analysis of Auto Rate-Tuning by Batch Normalization.
- [10] Gradient-Based Learning Applied to Document Recognition.
- [11] Stanford CS231n: Convolutional Neural Networks; Grace Hopper.
- [12] Speech and Language Processing.
- [13] Long Short-Term Memory; Learning to Forget: Continual Prediction with LSTM.
- [14] Understanding LSTM Networks.
- [15] PyTorch: autograd, Module, inference_mode, BCEWithLogitsLoss, CrossEntropyLoss, Conv2d, BatchNorm2d, Dropout, LSTM.